Used Car Price Classification Using Machine Learning

Project Overview

This project focuses on predicting used car price categories using several machine learning classification algorithms. Different models were trained and compared based on their performance using cross-validation and ROC-AUC scores. To improve the final model, hyperparameter tuning was performed with GridSearchCV, and the best model was saved as a reusable pipeline for future predictions.

Dataset

Dataset**:** Used Car Price Dataset

The dataset contains information about used vehicles, including:

- Brand
- Model
- Model Year
- Mileage
- Fuel Type
- Engine
- Transmission
- Exterior Colour
- Interior Colour
- Accident History
- Clean Title

Since the original target variable represented car prices, it was converted into a binary classification problem to simplify the prediction task.

Data Preprocessing

Before training the models, the dataset was preprocessed using the following steps:

- Missing numerical values were filled using the median with **SimpleImputer**.
- Categorical features were encoded into numerical values.
- The data was split into training and testing sets.
- Numerical features were standardized using **StandardScaler**.
- Binary class labels were created for supervised learning.

Machine Learning Models

The following classification models were implemented and evaluated:

- Logistic Regression

- Decision Tree (Default)
- Controlled Decision Tree
- Random Forest
- Gradient Boosting

Decision Tree Analysis

Default Decision Tree

The default Decision Tree was trained without restricting its depth (max_depth=None).

The model achieved very high accuracy on the training data but lower accuracy on the testing data, indicating overfitting. This happens because a fully grown decision tree can memorize the training data instead of learning patterns that generalize well to unseen data.

Controlled Decision Tree

To reduce overfitting, the following parameters were used:

- **max_depth = 5**
- **min_samples_split = 20**

Restricting the maximum depth prevents the tree from becoming overly complex, while increasing the minimum number of samples required to split a node reduces unnecessary splits caused by noise. Compared to the default tree, the controlled model showed a smaller gap between training and testing accuracy, indicating better generalization.

Gini vs Entropy

Two Decision Tree models were trained using different impurity measures.

Gini Impurity

$$\text{Gini} = 1 - \sum p_i^2$$

Entropy

$$\text{Entropy} = -\sum p_i \log_2(p_i)$$

A Gini impurity value of **0** indicates that all samples in a node belong to the same class, making it a perfectly pure node.

Test Accuracy

Gini Accuracy: <Enter Value>

Entropy Accuracy: <Enter Value>

Random Forest

Model Parameters

- n_estimators = 100

- max_depth = 10
- random_state = 42

Feature Importance

Random Forest estimates feature importance by measuring how much each feature reduces Gini impurity across all trees in the forest. Features with higher importance contribute more to accurate predictions.

Bagging

Random Forest uses **Bootstrap Aggregating (Bagging)** to improve performance.

- Each tree is trained on a randomly sampled subset of the training data (with replacement).
- At every split, only a random subset of features is considered.
- The final prediction is obtained by combining predictions from all trees.

This approach reduces overfitting and generally performs better than using a single decision tree.

Gradient Boosting

Model Parameters

- n_estimators = 100
- learning_rate = 0.1
- max_depth = 3

Feature Ablation Study

To understand the impact of less important features, the five least important features identified by the Random Forest model were removed and the model was trained again.

Interpretation

If the reduced model achieves a similar ROC-AUC score, the removed features contributed very little to the prediction and mainly added noise. However, if performance decreases significantly, those features contain useful information. Removing unnecessary features can also reduce model complexity, memory usage, and prediction time.

Cross-Validation

Five-fold Stratified Cross-Validation was performed using **ROC-AUC** as the evaluation metric.

Cross-validation provides a more reliable estimate of model performance because every sample is used for both training and validation across different folds. This reduces the dependency on a single train-test split and gives a better indication of how well the model is likely to perform on unseen data.

Hyperparameter Tuning

GridSearchCV was used to optimize the Random Forest pipeline.

Pipeline

- SimpleImputer

- StandardScaler
- RandomForestClassifier

Parameter Grid

- n_estimators
- max_depth
- min_samples_leaf

Best ROC-AUC

The search space included:

- 3 values for **n_estimators**
- 3 values for **max_depth**
- 2 values for **min_samples_leaf**

This resulted in **18 parameter combinations**.

Using **5-fold cross-validation**, GridSearchCV trained a total of **90 models**.

Although Grid Search requires more computation than Randomized Search, it evaluates every parameter combination and helps identify the best-performing model.

Manual Learning Curve

Training Data	Training AUC	Test AUC
20%	<Value>	<Value>
40%	<Value>	<Value>
60%	<Value>	<Value>
80%	<Value>	<Value>
100%	<Value>	<Value>

Interpretation

As more training data becomes available, training AUC usually decreases slightly because the model becomes less prone to overfitting. At the same time, the test AUC generally improves, showing better generalization. If the test AUC continues to increase even after using the full dataset, collecting additional data may further improve performance. If it levels off, model performance is more likely limited by the algorithm than by the amount of data.

Model Serialization

The best-performing pipeline was saved using:

joblib.dump(best_pipeline, "best_model.pkl")

The saved model was later reloaded using:

joblib.load("best_model.pkl")

Predictions on two unseen samples were successful, confirming that the saved pipeline can be reused without retraining.

Final Recommendation

Based on the evaluation results **Gradient Boosting Classifier** is recommended for deployment. It achieved the best balance between prediction accuracy, robustness, and generalization. Ensemble methods such as Random Forest and Gradient Boosting consistently outperformed a single Decision Tree by reducing overfitting and improving overall predictive performance. The optimized pipeline can also be saved and reloaded easily, making it suitable for deployment in real-world applications.